

Building the client fork

How to produce a playable build of this fork, what it costs, and the one decision that has to be made before anything is handed to anybody.

Measured on ENG-1, 2026-09-08. The server is a Go binary and its build is boring; this document is entirely about the Unity client.

Running it

```
$env:YARG_BUILD_OUTPUT = "C:\path\to\out\YARG.exe"   # optional; defaults under %TEMP%
$env:YARG_BUILD_DEVELOPMENT = "1"                    # optional; REQUIRED to reach a song server, see below
& "C:\Program Files\Unity\Hub\Editor\6000.3.5f2\Editor\Unity.exe" `
    -batchmode -nographics `
    -projectPath "C:\dev\YARG - Open Source Contributions\yarg" `
    -logFile "$env:TEMP\yarg-build.log" `
    -executeMethod YARG.Editor.BuildPlayerHeadless.Run
```

Launch it with `Invoke-CimMethod -ClassName Win32_Process -MethodName Create` rather than `Start-Process` — see `WORKSPACE.md` for why Unity dies otherwise when started from a bridge call. Grep the log for lines beginning `BUILD`.

To play against a song server, build with `YARG_BUILD_DEVELOPMENT=1`. The fork sets `insecureHttpOption` to `DevelopmentOnly` on purpose (see `ADR-004`), so a plain release build refuses `http://` — and a LAN song server is plain HTTP. Measured 2026-10-30: a release build pointed at a healthy server showed **"Song server: not reachable (Insecure connection not allowed)"** in the settings row, which reads like a network fault and is not one. A development build reaches it. What a *release* build should do is still the open question ADR-004 records.

Can the fork even be built? — measured 2026-09-08

This matters more than it sounds. The whole project's next gate is a public beta, and a beta means somebody who is not Jay running the client. Until 2026-09-08 nobody had ever produced a playable build of this fork; every measurement had been made either against the *official* v0.15.0 release (the oracle) or in editor batchmode. "It compiles headless" is not "it ships".

What upstream's own build needs, and what it does not tell us

Upstream has exactly one build workflow, `build-release-mac.yml`, and it is worth reading before assuming a Windows build is a solved problem:

- It uses `game-ci/unity-builder@v2.2.0` with `targetPlatform: StandaloneOSX`. **There is no Windows workflow at all**, so there is no upstream recipe to copy.
- It pins `unityVersion: 2021.3.21f1`. The project's `ProjectVersion.txt` says **6000.3.5f2**. That workflow is stale; do not treat its version as authoritative for anything.
- It installs **Blender 3.4.1** and runs **NuGetForUnity restore** before building.

Measured on ENG-1, because those two prerequisites decide whether a local build is even possible:

Blender	**not installed**
`.blend` files in `Assets`	**1**
`Assets/Packages` (NuGetForUnity)	**16 packages restored**
`Library/`	2.5 GB, already warm
Free disk	1.12 TB

The single `.blend` is already imported into the warm `Library`, so Blender is a **first-import** dependency rather than a build dependency. A machine with a cold `Library` would need it; this one does not. That distinction is the sort of thing that turns into a lost afternoon on a fresh checkout, so it is written down rather than left to be rediscovered.

`MakeTestBuild` cannot do it, and was not changed

The fork already has `Assets/Editor/MakeTestBuild.cs` from upstream, with *File → Make Test Build* and *Make Nightly Build* menu items. It cannot be driven from batchmode: it calls `BuildPlayerWindow.DefaultBuildMethods.GetBuildPlayerOptions(default)`, which asks the editor where to put the build, and in batchmode there is nobody to ask.

So `Assets/Editor/BuildPlayerHeadless.cs` is a **second entry point**, not an edit to that one. Upstream's menu items keep behaving exactly as upstream wrote them.

Two decisions inside it are worth stating:

- **Scenes come from `EditorBuildSettings`, not a hardcoded list.** A scene added to the project cannot then be silently missing from a build made this way. It found 6.
- **It exits non-zero when the build fails, and asserts the executable exists.** `BuildPipeline.BuildPlayer` returns a report rather than throwing, so a build that produced nothing at all still leaves the process exiting 0 unless somebody reads the result — and a build system that cannot go red is not a build system. It also prints every error message from every build step by name, because a summary count tells you the build is broken without telling you where to look.

The first build, measured

Unity 6000.3.5f2, ENG-1, `StandaloneWindows64`, warm `Library`, **cold shader cache**.

Wall time	**34.1 minutes**
Output	**526 MB**
Scenes	6
Warnings	103
Errors	**3** — see below
Scripting backend	Mono (not IL2CPP)

Nearly all of that time is shader compilation, and it is a first-run cost. Every pass logged `Local cache hits 0`; single passes compiled 2,880 variants for 6,142 seconds of CPU across 22 shader-compiler processes. It never saturated the machine — average CPU load was **17%** on 22 logical cores — so the box stayed usable throughout. A build runner with a cold cache should budget for this; a second build on the same machine should not pay it, and that is worth measuring rather than assuming.

"Success" came back with three errors, and that is the interesting part

Unity logged `Build Finished, Result: Success` **and** three errors:

```
BUILD ERROR [Preprocess Player]: RenderTexture.Create failed
BUILD ERROR [Preprocess Player]: RenderTexture.Create failed
BUILD ERROR [Building scene Assets/Scenes/Gameplay.unity]: RenderTexture.Create failed
```

Those are almost certainly `-nographics` artefacts — there is no graphics device in batchmode, so creating a render texture fails — and the resulting player looks complete. **"Almost certainly" is the honest strength of that claim**: nobody has launched this build. It is a judgement from the error text and the output, not a measurement.

What the run did prove is that the reporting was worth writing. `BuildPipeline.BuildPlayer` returns `Succeeded` with a non-zero error count, so a script that only checked the result would have called this clean. It now says `PASS WITH 3 ERROR(S)` and names each one with the build step it came from, because a green that has not been earned is worse than a red.

What is actually in the binary

Verified rather than assumed — the fork's own code is present in `Assembly-CSharp.dll` (2.1 MB): `SongServerSync`, `MirroredSongs`, `SongServerTab`, `WithServerBadge`, `BadgeMarkup`. So this is a build of *our* client, not a stock one that happened to compile.

The naming problem — settled 2026-09-08

The player used to identify itself as `companyName: YARC`, `productName: YARG` — upstream's values, because they are upstream's project settings and nothing had changed them. A binary handed to somebody was indistinguishable from the official build, so bugs we introduced would have looked like upstream's, and upstream would have fielded the reports.

It now builds as **`companyName: FatalException`**, **`productName: YARG-FE`**, **`bundleVersion: 0.1.0`** (fork commit `eff5a05b`).

The consequence that actually mattered was not the label. Unity derives `Application.persistentDataPath` from those two strings, so before the rename this fork wrote its `settings.json` and `songcache.bin` into `AppData/LocalLow/YARC/YARG` — the same folder an official install uses. A player running both got one set of settings and one cache, written by whichever binary ran last, with nothing on screen to explain it.

`Assets/Editor/IdentityProbe.cs` asserts the consequence rather than the cause: it reads `persistentDataPath` back out of a running editor and fails if it still lands under `/YARC/YARG`, if the build still claims upstream's name, **or if `PathHelper.LauncherPath` stopped pointing at the YARC Launcher.** That last check guards the opposite mistake — the launcher path is somebody else's install location, and if it ever follows our rename the fork can no longer read setlists the launcher put on disk. Measured on Unity 6000.3.5f2 batchmode:

```
companyName          = FatalException
productName          = YARG-FE
persistentDataPath   = .../AppData/LocalLow/FatalException/YARG-FE
LauncherPath         = ...\AppData\Local\YARC\Launcher
PASS
```

Still upstream's, deliberately: the window title and in-game branding. This change is about where bytes land, not what the splash screen reads.

Signing: decided against

Release binaries are **not code-signed and will not be** — see `docs/SYNC-CLIENT.md`. Free software does not carry a certificate subscription. Releases ship with `SHA256SUMS` and an honest warning in each archive's README, and the reproducible build means anyone who wants a signed binary can build and sign it themselves and prove it matches this source.

Putting a build on a second machine

Measured 2026-10-31, building for `MSI` (192.168.3.167) so a phone could drive the queue.

```
# ENG-1, any directory. Launch with Win32_Process.Create, not Start-Process (WORKSPACE.md).
# The .cmd sets YARG_BUILD_OUTPUT and YARG_BUILD_DEVELOPMENT=1 and calls Unity as above.
Invoke-CimMethod -ClassName Win32_Process -MethodName Create -Arguments @{ CommandLine = '"<path>\build-yarg.cmd"'
}
```

Result: 603 MB, 1.9 minutes, `BUILD RESULT: PASS WITH 2 ERROR(S)`**.** Both errors are `RenderTexture.Create failed`, which is `-nographics` and not a defect — the same class this file's own script documents from its first run. A build that reports PASS WITH ERRORS is still worth reading before trusting; these two were read.

The player is Mono, which matters more than it looks

`RemoteQueueServer` carries a KNOWN GAP saying its "binds `+:8099` without elevation" measurement was the editor, and that "a shipped player is IL2CPP with managed stripping". **For this project that premise is false.** `ProjectSettings.asset` sets `scriptingBackend` for `Android` only; Standalone has no override, so it builds Mono. Confirmed from the output tree rather than from the setting:

Evidence	Reading
`MonoBleedingEdge/` present	Mono
no `GameAssembly.dll`	not IL2CPP
34 `RemoteQueue` symbols in `YARG_Data/Managed/Assembly-CSharp.dll`	our code shipped

So the player is the same runtime family the editor probe passed on, and a development build does not strip. That is a much narrower gap than the comment implies — but it is still not a measurement **of the player**, and LAN mode is the half a phone needs. The first run on a second machine is that measurement. Since `679f00d3` the settings row reports a failed start, so a refused bind says so instead of leaving the setting reading `Lan`.

It cannot collide with an official YARG install

`companyName=FatalException`, `productName=YARG-FE`, so settings and songs live under `AppData/LocalLow/FatalException/YARG-FE`. An official YARG on the same machine keeps its own.

On the second machine

1. Copy the whole `build\msi\` folder across. `YARG.exe` needs the `YARG_Data` folder beside it; copying only the exe produces a launcher that does nothing useful. `YARG-FE_BurstDebugInformation_DoNotShip` can be left behind — its name is the instruction.
2. Run `YARG.exe`.
3. **Settings -> Song Server -> Song Server URL** = `http://192.168.2.7:8099`, then the sync button. The row says whether the server is reachable. If it reads "*Insecure connection not allowed*", the build is a release build and the wrong one — rebuild with `YARG_BUILD_DEVELOPMENT=1`.
4. **Settings -> Remote Queue =** `Lan`**.** The row then prints the address a phone should use. It binds `http://+:8099/`; Windows Firewall must allow inbound TCP 8099, and the first run may raise the firewall prompt.
5. Point the phone app at that address. The game's port is **8099**, the same number the song server uses on vault2 — different machines, so they do not collide, but do not read one address for the other.

`Lan` **has no authentication, by design and on purpose.** The server does not return 401 because there is nothing to authenticate as; it only refuses callers whose address is not private. Anyone on the network can queue and reorder songs. That is fine for a living room and not fine for a venue's open wifi.